Lenet on MNIST

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import sequential
from tensorflow.keras.layers import Conv2D, AvgPool2D,
    Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical, plot_model
import matplotlib.pyplot as plt
(train_x, train_y), (test_x, test_y) = mnist.load_data()
print("The size of train_x is: {}".format(train_x.shape))
print("The size of train_y is {}".format(train_y.shape))
print("The size of test_x is {}".format(test_x.shape))
print("The size of test_y is {}".format(test_y.shape))
train_x = train_x.reshape(train_x.shape[0], 28,28,1)
test_x = test_x.reshape(test_x.shape[0], 28, 28,1)
train_x = train_x/255.0
test_x = test_x/255.0
train_y = to_categorical(train_y, num_classes=10)
test_y = to_categorical(test_y, num_classes=10)
model = sequential()
model.add(Conv2D(filters=6, kernel_size=(5,5),
        padding='valid', input_shape=(28,28,1),
    activation='tanh'))
model.add(AvgPool2D(pool_size=(2,2)))
model.add(Conv2D(filters=16, kernel_size=(5,5),
        padding='valid', activation='tanh'))
model.add(AvgPool2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(120, activation='tanh'))
model.add(Dense(84, activation='tanh'))
model.add(Dense(10, activation='softmax'))
model.summary()
model.compile(loss='categorical_crossentropy',
    optimizer=Adam(), metrics=['accuracy'])
test_loss_list=[]
test_accuracy_list=[]
for i in range(20):
    print("\n Iteration
```

```
model.fit (train_x, train_y), batch_size =128, epochs=1,
  verbose = 1, validation_data=(test_x, test_y))
score = model.evaluate (test_x, test_y, verbose=0)
print ("Test Loss", score [0])
print ("Test Accuracy", score [1])
test_loss_list.append (score[0])
test_accuracy_list.append (score[1])
print ("\n Test loss for 20 iterations.", test_loss_list)
print ("Test Accuracy for 20 iterations:" test_accuracy_list)
conv_layer = model.layers[0]
filters = conv_layer get_weights()[0]
plt.figure (figsize=(10,10))
for i in range(6):
    plt.subplot (4,8,i+1)
    plt.imshow (filters [:,:,0,i],cmap='gray')
    plt.axis('off')
plt.suptitle ('CNN Filters After Training')
plt.tight_layout()
plt.show()
```

OUTPUT

```
The size of train_x is: (60000,28,28)
The size of train_y is: (60000,)
The size of test_x is: (10000, 28,28)
The size of test_y is: (10000,)
Model: "sequential_1"
Layer (type)                    output shape          Param #
conv2d_2 (Conv2D)               (None, 24,24,6)       156
average_pooling2d_2             (None, 12,12,6)       0
(Average Pooling2D)
Conv2d_3 (Conv2D)               (None, 8,8,16)        2.416
average_pooling2d_3             (None, 4,4,16)        0
(Average Pooling 2D)
flatten_1 (flatten)             (None, 256)           0
dense_3 (dense)                 (None, 120)           30.840
dense_4 (dense)                 (None, 84)            10.164
dense_5 (Dense)                 (None, 10)            850
Total params: 44.426 (173.54 KB)
Trainable params: 44.426 (173.54 KB)
Non-trainable params: 0 (0.00B)
```